EGC 211P Mid-sem help sheet

Kushman

1. Class from UML

```
class Item {                          // UML box -> class
    static int uniq;                  // underlined -> static
    static const double LIMIT;        // const static (non-int)
    int id;                           // "- id: int"  -> private
    string name;
protected:                            // "# x" -> protected
    double price;
public:                               // "+ f()" -> public
    Item(const string& n, double p) : id(++uniq), name(n), price(p) {}
    virtual ~Item() {}                // ANY virtual -> virtual dtor
    int getID() const { return id; } // "f() const: int"
    virtual double cost() const { return price; }   // italic -> virtual
    virtual void show() const = 0;    // {abstract}/italic class -> pure
};
int Item::uniq = 0;                   // define statics ONCE, outside
const double Item::LIMIT = 100.0;
```

2. Inheritance / override

```
class Book : public Item {            // hollow triangle -> public inheritance
    string author;
public:
    Book(const string& n, double p, const string& a)
        : Item(n, p), author(a) {}    // base ctor FIRST in init list
    double cost() const override { return Item::cost() * 1.05; }
    void show() const override {
        cout << getID() << " " << fixed << setprecision(2) << cost() << "\n";
    }
};
```

3. Operators

```
class Wallet {
    double bal = 0.0;
public:
    Wallet& operator+=(double a) { bal += a; return *this; }
    Wallet& operator+(double a)  { bal += a; return *this; } // member, returns ref
    Wallet& operator++()    { ++bal; return *this; }        // ++w (prefix)
    Wallet  operator++(int) { Wallet t = *this; ++bal; return t; } // w++
    bool operator==(const Wallet& o) const { return bal == o.bal; }
    friend ostream& operator<<(ostream& os, const Wallet& w) { return os << w.bal; }
};
```

4. Exceptions

```
class AppException : public runtime_error {
public:
    AppException(const string& m) : runtime_error(m) {}
};
class NotFound : public AppException {
public:
    NotFound(const string& m) : AppException(m) {}
};
void demoExc() {
    try { throw NotFound("x"); }
    catch (NotFound& e)     { cout << e.what(); }   // most-derived FIRST
    catch (AppException& e) { cout << e.what(); }
    catch (int code)        { cout << code; }       // throw 42;
    catch (...)             { throw; }              // rethrow
}
```

5. enum class + names

```
enum class Status { Open, Closed };                 // values 0, 1
const vector<string> statusName = {"Open", "Closed"};
string toStr(Status s) { return statusName[static_cast<int>(s)]; }
Status fromInt(int i) { return static_cast<Status>(i); }
```

6. STL / strings

```
void demoStl(vector<Item*>& v, map<string, int>& m) {
    for (Item* p : v) if (p->getID() == 3) { /* found */ }
    for (int i = (int)v.size() - 1; i >= 0; i--) v[i]->show(); // reverse
    v.erase(v.begin() + 1);                         // remove index 1
    auto it = find_if(v.begin(), v.end(), [](Item* p){ return p->getID() == 2; });
    if (it != v.end()) (*it)->show();
    m["a"]++;  if (m.count("a")) {}  for (auto& kv : m) cout << kv.first << kv.second
    ;
    string s = "Rs." + to_string(5.0);              // "Rs.5.000000" (6 dp!)
    ostringstream os; os << 5.0; s = os.str();      // "5"
}
```

7. UML → C++

`-` / `#` / `+` (red □ / yellow ◇ / green ○)	private / protected / public
underlined	`static` (+ define outside class)
italic method / class	`virtual` / abstract (`=0` somewhere)
`f(p: T) const: R`	`R f(T p) const;`
hollow triangle ◁	`class D : public B`
filled diamond ◆ (composition)	owns: member object / `vector<T>`
hollow diamond ◇ (aggregation)	holds: `T*` / `vector<T*>`
arrow / line + 1, *	pointer member; * ⇒ vector

8. Pitfalls that cost marks

- `extern const vector<string> X;` in .h but `vector<string> X` in .cpp ⇒ **compile error** (2025 solution!).
- Non-void method without `return` compiles under `-w`, returns garbage. Compile with `-Wall -Wextra`.
- Every member initialised in *every* ctor (status!). `= default` leaves ints/enums garbage.
- Static member defined exactly once, outside the class: `int A::uniq = 0;`
- `vector<T>` stores copies: mutating the copy is lost. Need shared state ⇒ `vector<T*>`.
- const method can only call const methods. Getters: always `const`.
- `;` after } of class. Class used before defined ⇒ follow question order.
- `fixed`/`setprecision` stay on `cout` for the rest of the run.
- `to_string(double)` gives 6 decimals. Copy-paste format strings; never retype (spaces!).
- Check-order tables: implement checks in exactly the listed order (first match wins).
- Base ctor runs first; virtual call inside a ctor is NOT polymorphic.

9. Exam protocol

1. 15 min: read whole pdf, annotate check-order tables, then write all class shells + empty bodies ⇒ compile (UML marks banked).
2. One method at a time, compile after each. Run local tester after each class.
3. `g++ -std=c++17 -Wall -Wextra skeleton.cpp`
4. T−10 min: stop. Comment out anything that doesn't compile. Submit.

10. My mistakes (fill in after the mock)